Vector Database Architecture and Indexing

Vector databases employ sophisticated indexing structures to enable fast similarity search across millions or billions of embeddings. Approximate Nearest Neighbor (ANN) algorithms like HNSW (Hierarchical Navigable Small World), LSH (Locality Sensitive Hashing), and IVF (Inverted File Index) replace exhaustive search with efficient approximations. HNSW constructs hierarchical graphs connecting vectors based on proximity, enabling logarithmic-time search through graph traversal. LSH hashes similar vectors to the same buckets, reducing search space significantly. IVF clusters vectors into partitions, searching only relevant partitions rather than the entire index. Quantization techniques compress vectors, reducing memory requirements and increasing search speed at the cost of slight accuracy trade-offs. Product quantization divides vectors into subspaces, compressing each independently. Scalar quantization reduces bit precision. Indexing strategies balance memory usage, search speed, and accuracy. Most vector databases support multiple index types, allowing optimization for specific use cases. Real-time indexing accommodates dynamically inserted embeddings without full reindexing. Metadata filtering enables searching within subsets of vectors matching certain criteria. Partitioning and sharding distribute data across multiple nodes for scalability. Understanding indexing choices is crucial for optimizing RAG system performance.